Compiladores e Intérpretes

Tema 4.4 – Analizador Semántico – Chequeo de Declaraciones

Universidad Nacional del Sur
Departamento de Ciencias e Ingeniería de la Computación
2026

Chequeo de Declaraciones - Intuición

- El analizador **semántico** tomará las **estructuras generadas** por el sintáctico y
  - Terminará de **recopilar la información** del programa y las **relaciones** entre las **entidades**
  - **Controlar** que el programa es **semánticamente valido**
- Es usual agrupar estas tareas según el nivel donde se realizan...

Chequeo de Declaraciones

Chequeo de Sentencias

Chequeo de Declaraciones - Intuición

- El analizador **semántico** tomará las **estructuras generadas** por el sintáctico y
  - Terminará de **recopilar la información** del programa y las **relaciones** entre las **entidades**
  - **Controlar** que el programa es **semánticamente valido**
- Es usual agrupar estas tareas según el nivel donde se realizan...

Chequeo de Declaraciones

Se focaliza solamente en las declaraciones. Es decir en controlar que toda declaración sea semanticamente valida y en recopilar la información que nos falte de las entidades que se declararon

Chequeo de Declaraciones - Intuición

- En un lenguaje como MiniJava podemos dividir fácilmente el programa en declaraciones y sentencias
  - Todo lo que esta **dentro** de un **bloque** es una **Sentencia** (incluyendo el bloque), es decir las cosas que se “ejecutan” en nuestro programa
  - Todo lo que esta **fuera** de un **bloque** es una **Declaración**, es decir, las cosas que nos permiten definir como son las/los:
    - **Clases**
    - **Atributos**
    - **Métodos**

Chequeo de Declaraciones - Intuición

- Las tareas de chequeo de declaraciones se realizan sobre en general sobre la Tabla de Símbolos
  - Por lo tanto es necesario **realizar el chequeo de declaraciones una vez finalizado el análisis sintáctico** (es decir en una segunda pasada)
  - Aun así, es posible que **alguna partecita** de estas tareas convenga realizarse **mientras** que se **recopila** la información en el **sintáctico** (como por ejemplo algunos controles de **nombres repetidos**)

Chequeo y Consolidación de Declaraciones en MiniJava

Chequeo de Declaraciones en MiniJava

- El **chequeo de declaraciones** tiene dos actividades principales:
  1) **Chequear** que toda entidad fue **correctamente declarada**
  2) **Consolidar** la Tabla de Simbolos

- Estas actividades dependen de las restricciones de declaración del lenguaje
  - Caso de estudio: MiniJava

Chequeo de Declaraciones en MiniJava

- Chequear que **toda declaración** fue

Correctamente Declarada

- En general implica chequear que:
  - No haya **nombres repetidos** en el mismo contexto (*ej:* dos clases con el mismo nombre)
  - Todo **nombre usado** en una declaración **haya sido declarado** en el contexto adecuado (*ej:* el tipo un atributo o retorno de un método)
  - Custiones propias a cada **entidad** (herencia circular, redefinición, etc)

Chequeo de Declaraciones en MiniJava

- Chequear que **toda declaración** fue

Correctamente Declarada

- En general implica chequear que:
  - No haya **nombres repetidos** en el mismo contexto (*ej:* dos clases con el mismo nombre)
  - Todo **nombre** ción **haya sido declarado** en el contexto adec ibuto o retorno de un método)
  - Custiones pro erencia circular, redefinición, etc)

puede realizarse **DIRECTAMENTE**, mientras se construye la TS en el Análisis Sintáctico!

Chequeo de Declaraciones en MiniJava

- Los controles y tareas se realizarán **directamente** sobre la **Tabla de Símbolos**
- Para esto se analizarán las entradas de
  - Clases concretas
  - Métodos
  - Constructores
  - Atributos y Parámetros

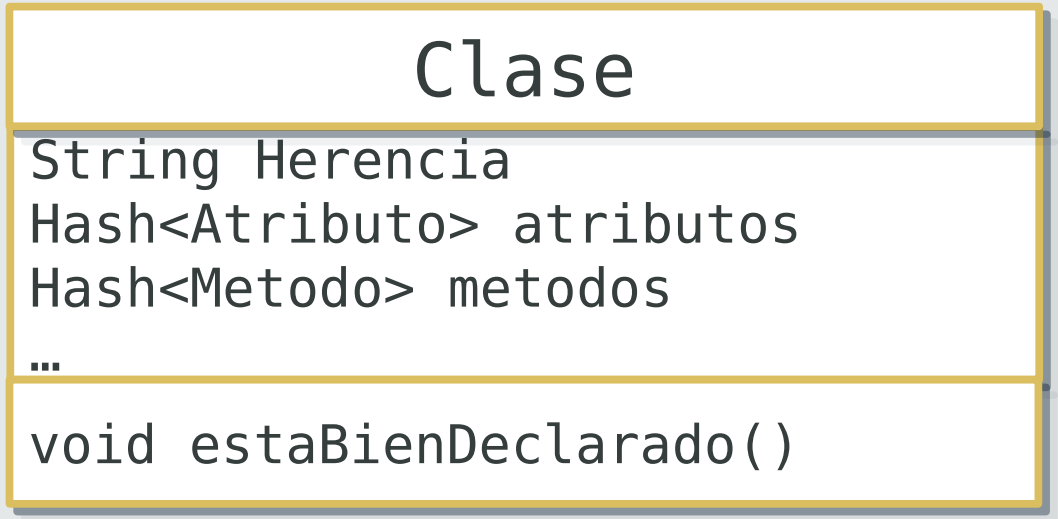
Los controles de **Variables locales** deben realizarse en el **Chequeo de Sentencias**!!!

Chequeo de Declaraciones en MiniJava

- Por ejemplo, para la **entrada de una clase concreta** en la TS se controla que:
  - Que todos sus **métodos**, sus **variables** de instancia y su **constructor** se encuentren **correctamente declarados**
  - Si tiene herencia explícita **herede de una clase/interface declarada**
  - **No** tenga **herencia circular** (*i.e.* que en su línea de ancestros no aparezca ella misma)
  - Que **respete adecuadamente** genericidad
  - Que implemente todos los **metodos** de su ancestro si hereda de una **interface**
  - Otros...

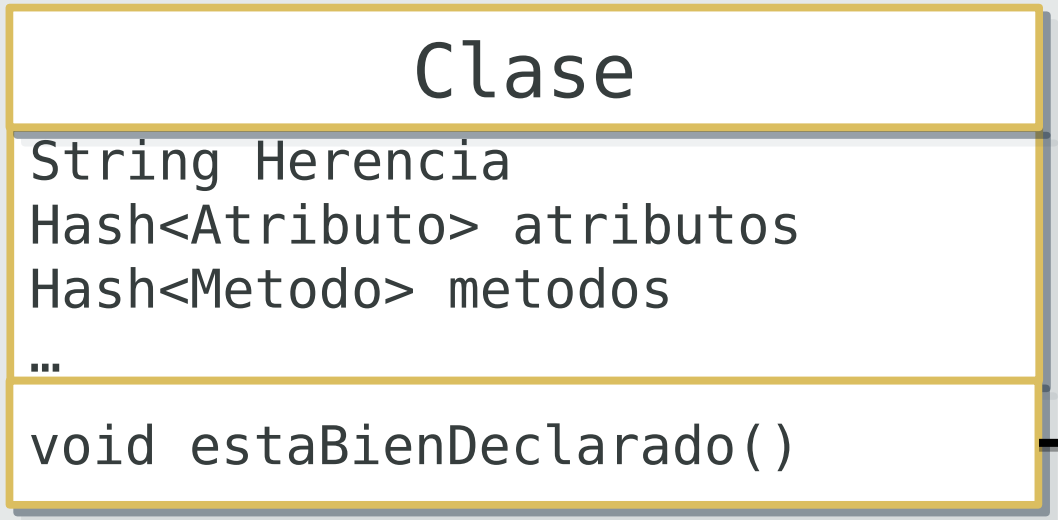
Chequeo de Declaraciones en MiniJava

- La **implementación** de todos estos **controles** puede diseñarse agregando **métodos** para realizar los controles directamente en las **entradas** de la TS



Chequeo de Declaraciones en MiniJava

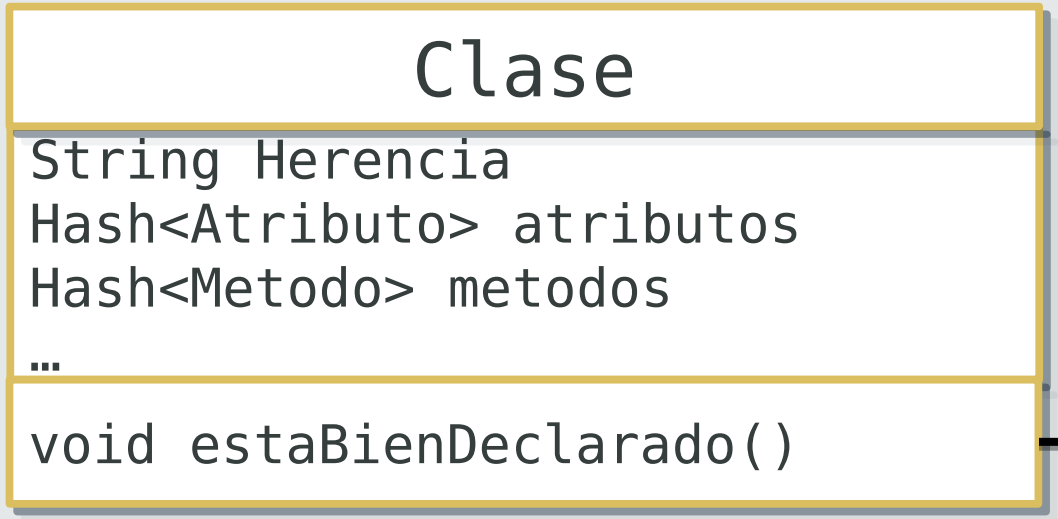
- La **implementación** de todos estos **controles** puede diseñarse agregando **métodos** para realizar los controles directamente en las **entradas** de la TS




```
Para cada var en atributos
    var.estaBienDeclarada()
Para cada met en metodos
    met.estaBienDeclarado()
Si no(TS.ExisteClase(Herencia))
    entonces ERROR!
this.checkircularidad()
…
```

Chequeo de Declaraciones en MiniJava

- La **implementación** de todos estos **controles** puede diseñarse agregando **métodos** para realizar los controles directamente en las **entradas** de la TS

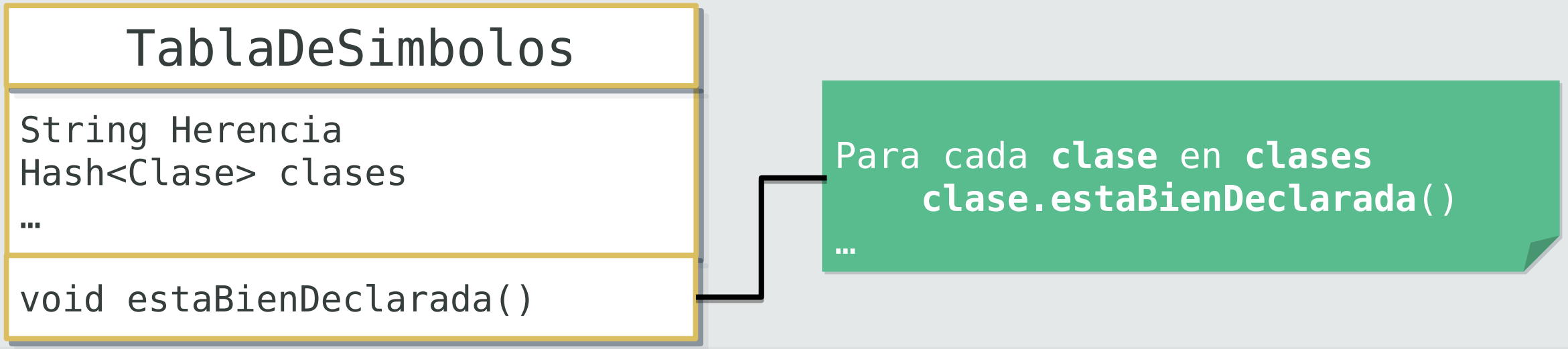



```
Para cada var en atributos
    var.estaBienDeclarada()
Para cada met en metodos
    met.estaBienDeclarado()
Si no(TS.ExisteClase(Herencia))
    entonces ERROR!
this.checkircularidad()
…
```

Puede haber que controlar mas cosas!!!!

Chequeo de Declaraciones en MiniJava

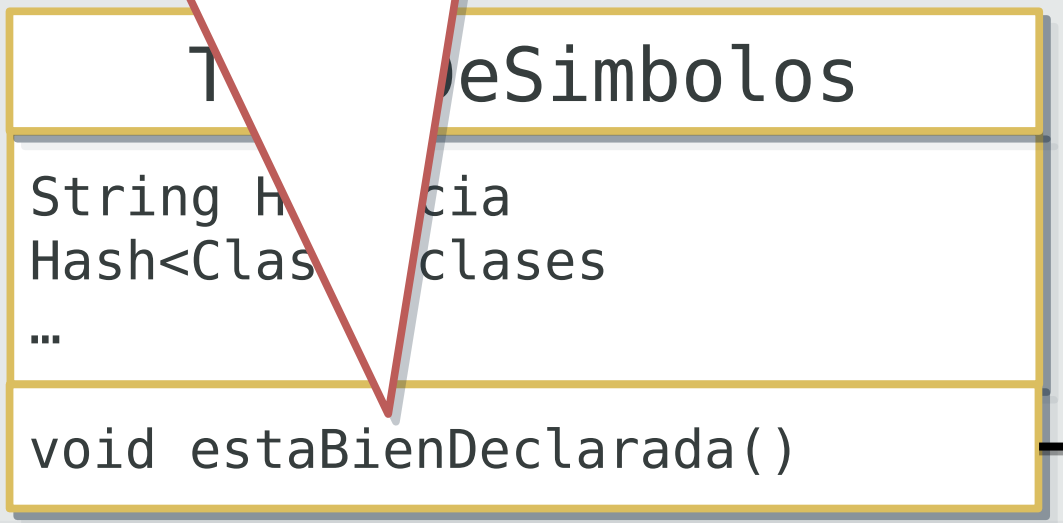
- El chequeo de que cada entidad esta bien declarada **arranca** desde la **tabla de símbolos**



Chequeo de Declaraciones en MiniJava

- esta bien declarada **arranca**

Atencion! Es muy importante que antes de arrancar con esto, todas las clases predefinidas (con sus miembros) hayan sido agregadas a la TS!!!




```
Para cada clase en clases
    clase.estaBienDeclarada()
…
```

Chequeo de Declaraciones en MiniJava
Consolidación

Consolidación de la TS

- **Otra tarea importante** que debe realizarse en el chequeo de declaraciones es la

Consolidación de Clases

- Consiste en actualizar las tablas con las entidades que son heredadas de otras clases
- Esto implica agregar (adecuadamente)
  - todos los métodos heredados (consolidación de tablas de métodos)
  - todos atributos heredados (consolidación de tablas de atributos)

Consolidación de la TS

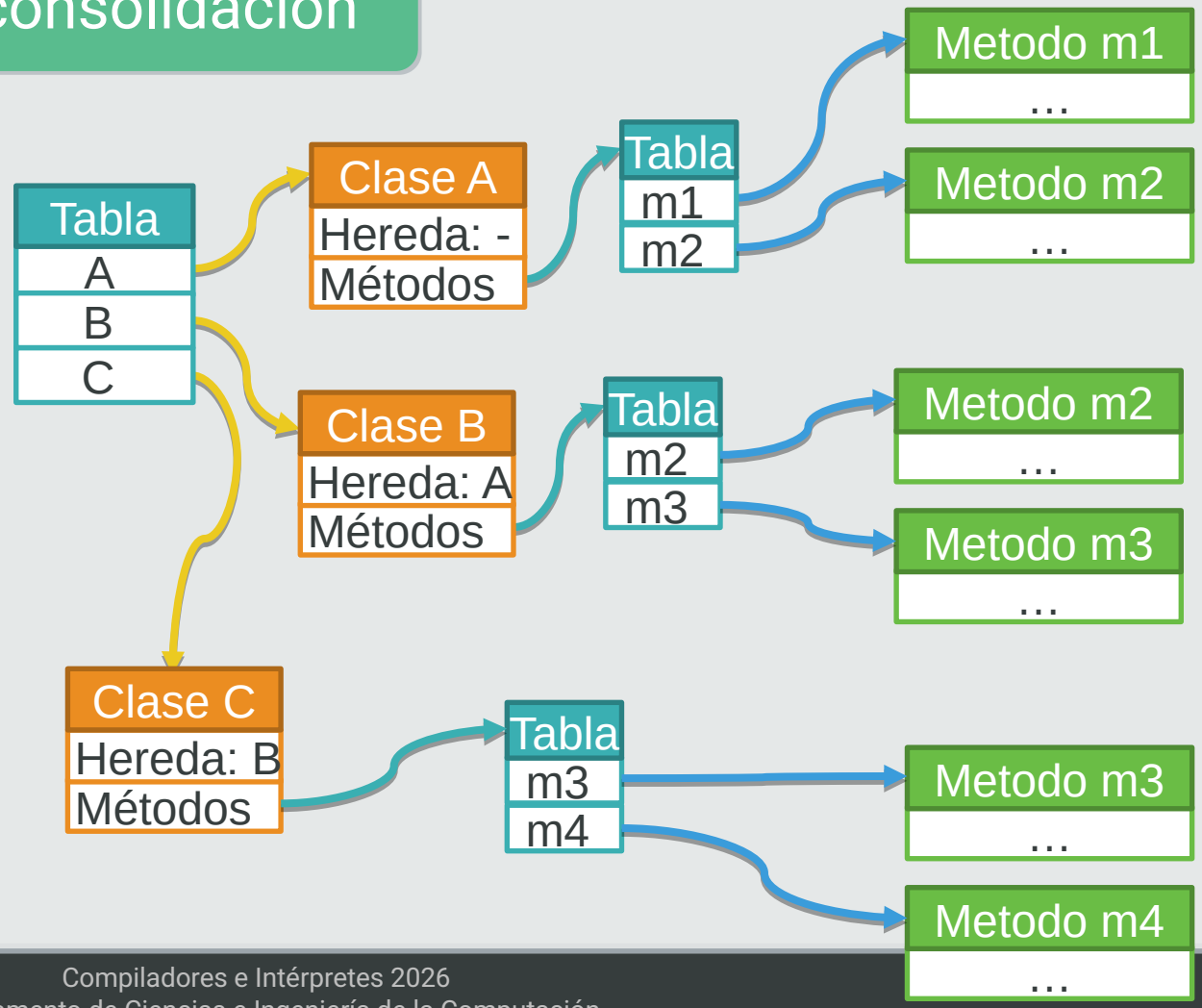
- Consolidación de tablas de métodos
- Consideraciones:
  - **Métodos redefinidos**: una clase sólo tiene acceso a la **última versión** de los métodos que tiene y hereda
  - Controlar que todos los metodos de interface estan definidos
  - Además, **controlamos:**
    - si un método es **correctamente redefinido**
    - si la **clase implementa** todos los **métodos abstractos** si su ancestro es una interfaz
    - Acomodar la genericidad

Consolidación de la TS

Tablas de Métodos antes de la consolidación

```
class A {
    dynamic void m1(){…}
    dynamic void m2(int p){…}
}
class B extends A{
    dynamic void m2(int x){…}
    dynamic void m3(){…}

}
class C extends B{
    dynamic void m3(){…}
    dynamic void m4(){…}
}
```

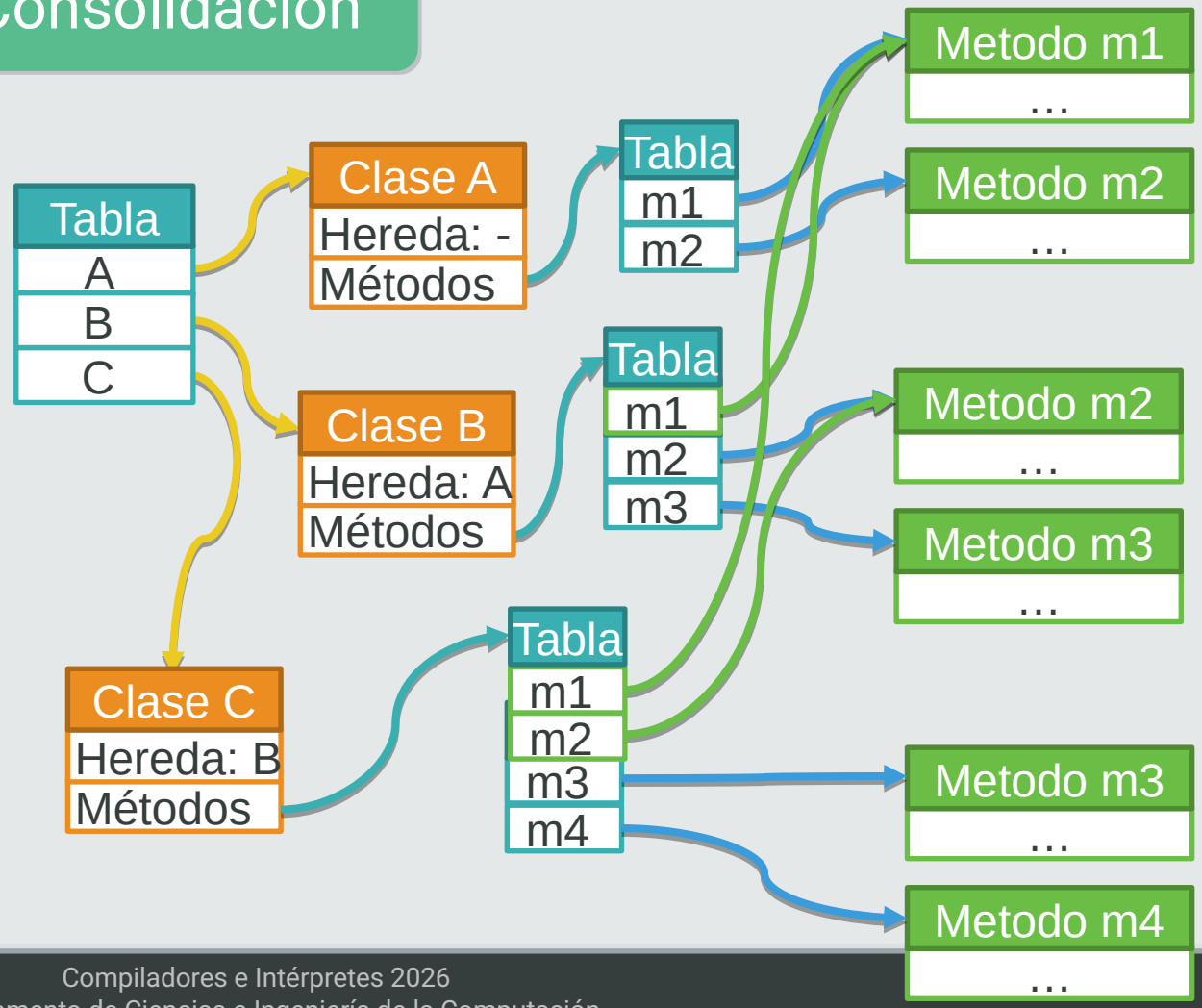


Consolidación de la TS

Tablas de Métodos luego de la Consolidación

```
class A {
    dynamic void m1(){…}
    dynamic void m2(int p){…}
}
class B extends A{
    dynamic void m2(int x){…}
    dynamic void m3(){…}

}
class C extends B{
    dynamic void m3(){…}
    dynamic void m4(){…}
}
```



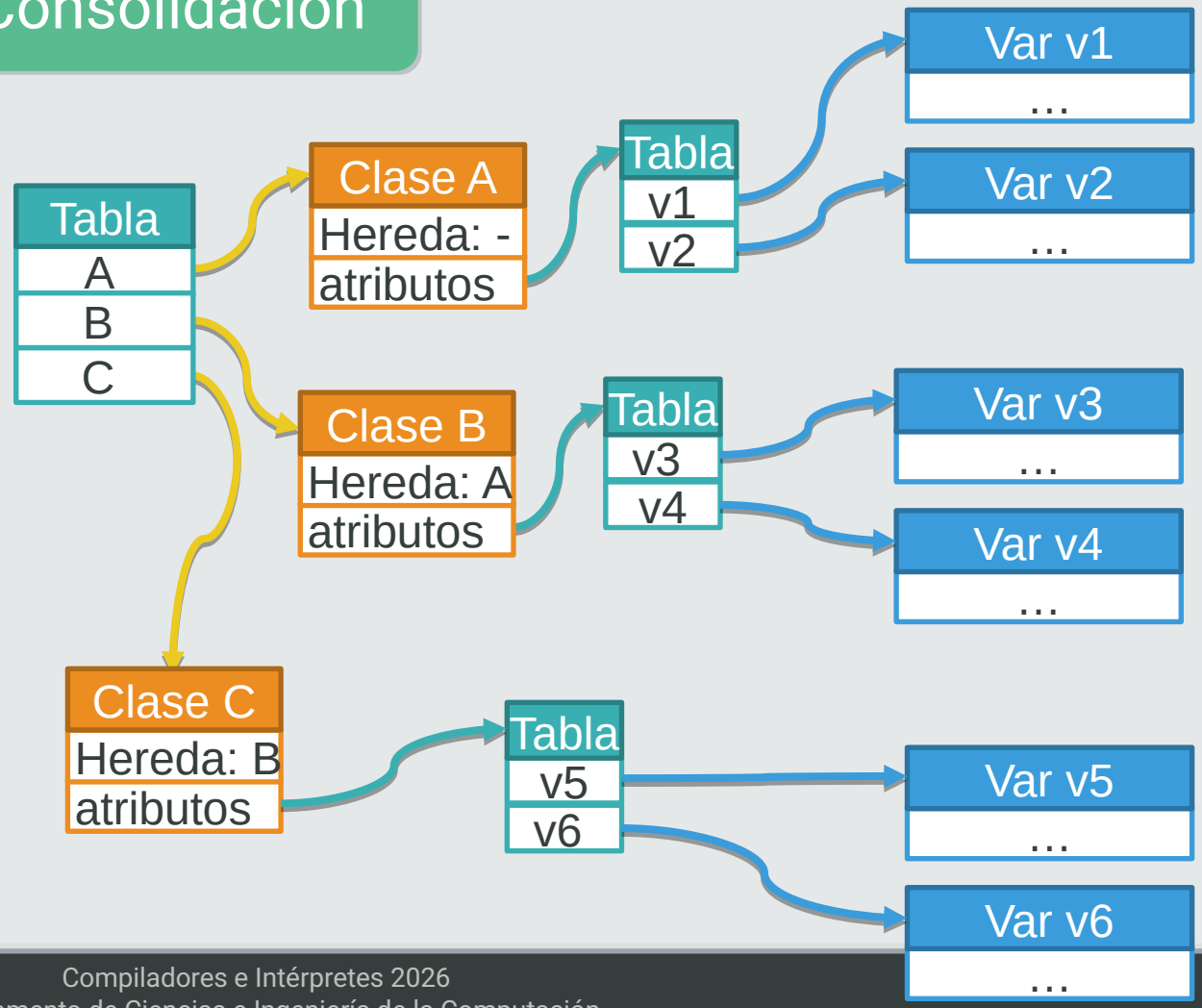
Consolidación de la TS

- **Consolidación** de tablas de **Atributos**
- Consideraciones:
  - Podemos aprovechar la actualización de la tabla para **controlar** que no se haya **declarado un atributo** con el **mismo nombre** que el de **un ancestro**

Consolidación de la TS

Tablas de Atributos antes de la Consolidación

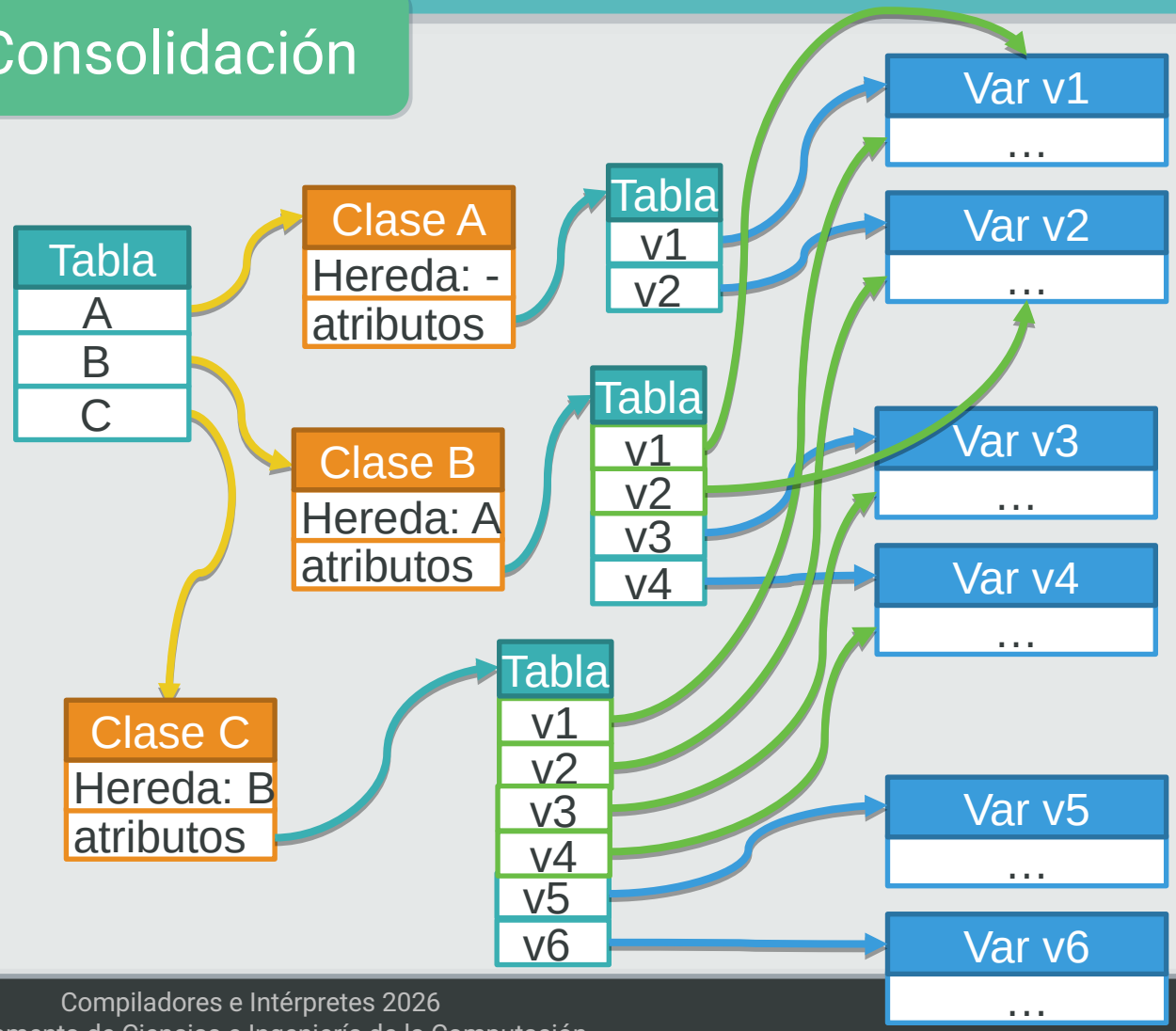
```
class A {
    int v1
    int v2
}
class B extends A{
    String v3
    boolean v4
}
class C extends B{
    int v5
    int v6
}
```



Consolidación de la TS

Tablas de Atributos luego de la Consolidación

```
class A {
    int v1
    int v2
}
class B extends A{
    String v3
    boolean v4
}
class C extends B{
    int v5
    int v6
}
```



Consolidación de la TS

- En general, para realizar los procesos de **Consolidación** nos vamos a valer de la **Delegación** y **Recursión**
- La intuición es que una **Clase** puede consolidarse solo si su **Ancestro** directo esta **consolidado**, **sino** le pedirá que a este que se **consolide** y luego realizara su consolidación
- Es recomendable manejar la consolidación de **atributos separada** de la **métodos**

Como y Cuando Arranca el Semantico?

- Siempre **arranca** después de **terminar** el análisis **sintáctico**
  - Cuando hacemos el **Chequeo de declaraciones**?
  - Cuando hacemos la **Consolidación**?
  - Como se **inicializa** todo?



Errores – Chequeo de Declaraciones

- Los **errores semánticos** en el chequeo de declaraciones se producen cuando se detecta una entidad mal declarada
- En MiniJava, en general, los **errores** se producen cuando al **declarar una entidad** utilizamos una **clase no declarada**
- Pero también, se pueden producir por **métodos mal redefinidos**, **nombres repetidos**, **herencia circular**, entre otros…

Errores – Chequeo de Declaraciones

- Para **reportar los errores** es necesario indicar en que **numero de línea** se produce el error
- En la **segunda pasada** ya **no tenemos los tokens**
- Por lo tanto la **entidades** (y sus respectivas entradas) deben **llevar cuenta del numero de línea** donde fueron declaradas
  - Usualmente se determina con el **token del identificador** utilizado para nombrarlas